

**Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών
Υπολογιστών**

Αριστοτέλειο Πανεπιστήμιο Θεσσαλονίκης



Νευρωνικά Δίκτυα – Βαθιά Μάθηση
Εργασία 1

Τζίνα Θεοδώρα (ΑΕΜ: 10715)

Περιεχόμενα

1	Εισαγωγή	3
2	Μέρος Α – Προεπεξεργασία	3
2.1	K-Nearest Neighbors	4
2.2	Nearest Class Centroid	11
2.3	Σύγκριση – Συμπεράσματα	14
3	Μέρος Β – Convolutional Neural Network	15
3.1	Architecture and Hyperparameters	15
3.2	Model Training	18
3.3	Σύγκριση – Συμπεράσματα	25
4	Μέρος Γ – Multilayer Perceptron	25

1 Εισαγωγή

Στην παρούσα εργασία καλούμαστε να υλοποιήσουμε ένα νευρωνικό δίκτυο εμπρόσθιας τροφοδότησης (feedforward NN), το οποίο θα εκπαιδεύεται με τον αλγόριθμο back-propagation. Το νευρωνικό δίκτυο που θα υλοποιηθεί, καλείται να επιλύει οποιοδήποτε πρόβλημα κατηγοριοποίησης πολλών κλάσεων με επιβλεπόμενη ή αυτό-επιβλεπόμενη μάθηση. Στο πρώτο μέρος της εργασίας, γίνεται η υλοποίηση δύο «θεμελιωδών» μεθόδων κατηγοριοποίησης: K-Nearest Neighbors και Nearest Class Centroid, ενώ στο δεύτερο μέρος ακολουθεί η υλοποίηση του εκτενέστερου νευρωνικού μοντέλου.

Για την διεκπεραίωση της εργασίας επιλέχθηκε το [CIFAR-10 dataset](#), το οποίο αποτελείται από 60.000 έγχρωμες εικόνες των 32x32 pixels x 3 RGB color channels, ταξινομημένες σε 10 κλάσεις, με 6.000 εικόνες ανά κλάση: αεροπλάνο, αυτοκίνητο, πουλί, γάτα, ελάφι, σκυλί, βάτραχος, άλογο, πλοίο, φορτηγό. Συνολικά υπάρχουν 50.000 εικόνες εκπαίδευσης και 10.000 εικόνες δοκιμής. Οι διαστάσεις των training και test sets διαμορφώνονται αρχικά ως εξής:

- Training data shape: (50000, 32, 32, 3)
- Training labels shape: (50000, 1)
- Test data shape: (10000, 32, 32, 3)
- Test labels shape: (10000, 1)

2 Μέρος Α - Προεπεξεργασία

Αρχικά γίνεται εισαγωγή του dataset μέσω της βιβλιοθήκης Keras της Tensorflow, το οποίο αποτελείται από εικόνες όπου κάθε χρωματικό κανάλι (Κόκκινο, Πράσινο, Μπλε) χρησιμοποιεί 8 bits, δηλαδή $2^8 = 256$ διαφορετικές τιμές. Οι τιμές αυτές υπόκεινται σε κανονικοποίηση με εύρος $[0, 255] \rightarrow [0, 1]$. Έπειτα, πραγματοποιείται υποβάθμιση των διαστάσεων των δεδομένων έτσι ώστε να είναι δυνατή η μετέπειτα επεξεργασία τους από τα μοντέλα ταξινόμησης. Συγκεκριμένα, η εντολή `x_train = x_train.reshape((x_train.shape[0], -1))` μετατρέπει τις εικόνες σε 2D μορφή όπου η πρώτη παράμετρος `x_train.shape` δηλώνει τον αριθμό των δειγμάτων, ενώ το `-1` υποδηλώνει την ισοπέδωση όλων των pixels στη δεύτερη διάσταση. Αντίστοιχα, η εντολή `y_train = y_train.flatten()`, ισοπεδώνει τα labels των εικόνων σε μία μόνο διάσταση:

- Processed training data shape: (50000, 3072)
- Processed test data shape: (10000, 3072)

Απαραίτητη για την ανώδυνη διεκπεραίωση της εργασίας κρίθηκε η περεταίρω μείωση των διαστάσεων των δεδομένων μέσω Principal Component Analysis (PCA), μία μέθοδος η οποία μετατρέπει τα αρχικά δεδομένα σε ένα νέο σύστημα συντεταγμένων, διατηρώντας τα σημαντικότερα. Το PCA υπολογίζει τα ιδιοδιανύσματα (eigenvectors) του πίνακα συνδιακύμανσης, τα οποία αντιπροσωπεύουν τις κατευθύνσεις με τη μεγαλύτερη διακύμανση στα δεδομένα. Για την επιλογή του πλήθους των principal components, εξετάστηκαν οι τιμές 50, 100, 200, οδηγώντας σε παρόμοια αποτελέσματα, οπότε διατηρήθηκε η αρχική τιμή $n_components = 50$ αφού προφανώς προσφέρει και την πιο γρήγορη ταξινόμηση μετέπειτα:

- Applying PCA: 3072 $\rightarrow$ 50 dimensions
- Explained variance: 84.30%
- New training shape: (50000, 50)
- New test shape: (10000, 50)

Η προεπεξεργασία αυτή, βρίσκεται στο αρχείο *dataProccessing.py*.

2.1 K-Nearest Neighbors

Ο αλγόριθμος K-Nearest Neighbors (kNN) βασίζεται στην ιδέα ότι δεδομένα που βρίσκονται κοντά μεταξύ τους, πιθανώς ανήκουν και στην ίδια κλάση.

1. **Επιλογή k:** πρόκειται για το πλήθος των «γειτόνων», οι οποίοι θα συμβάλουν στην κατηγοριοποίηση του δείγματος σε κλάση. Στην συγκεκριμένη εργασία κληθήκαμε να εξετάσουμε για $k = 1$, $k = 3$.
2. **Υπολογισμός απόστασης:** υπολογίζεται από το νέο δείγμα μέχρι και όλα τα σημεία του συνόλου εκπαίδευσης. Παρακάτω εξετάστηκαν διάφοροι τύποι υπολογισμού αυτής της απόστασης.
3. **Εύρεση k γειτόνων:** ταξινόμηση των παραπάνω αποστάσεων και επιλογή των k μικρότερων, χαρακτηρίζοντας τα αντίστοιχα δείγματα ως k πλησιέστεροι γείτονες.
4. **Κατηγοριοποίηση:** το νέο δείγμα εντάσσεται στην κλάση που υποδεικνύει η πλειοψηφία των k πλησιέστερων γειτόνων.

Αξίζει να σημειωθεί πως η μέθοδος αυτή χαρακτηρίζεται ως “Lazy learner”, αφού δεν εκπαιδεύεται στην πραγματικότητα από συγκεκριμένους κανόνες, αλλά απλώς αποθηκεύει δεδομένα. Αυτό προοικονομεί τα ανεπαρκή αποτελέσματα που θα ακολουθήσουν, αφού συμπεραίνουμε ότι δεν αποτελεί τη βέλτιστη λύση για μεγάλα σύνολα δεδομένων.

Το μοντέλο χτίστηκε πάνω στο *training set*, εξετάστηκε τόσο στο *training* όσο και στο *test set*, αλλά δόθηκε βάση στην μελέτη των *test set* αποτελεσμάτων. Η υλοποίησή του βρίσκεται στο αρχείο *kNearestNeighbors.py*.

➤ **Euclidean distance, k = 1:**

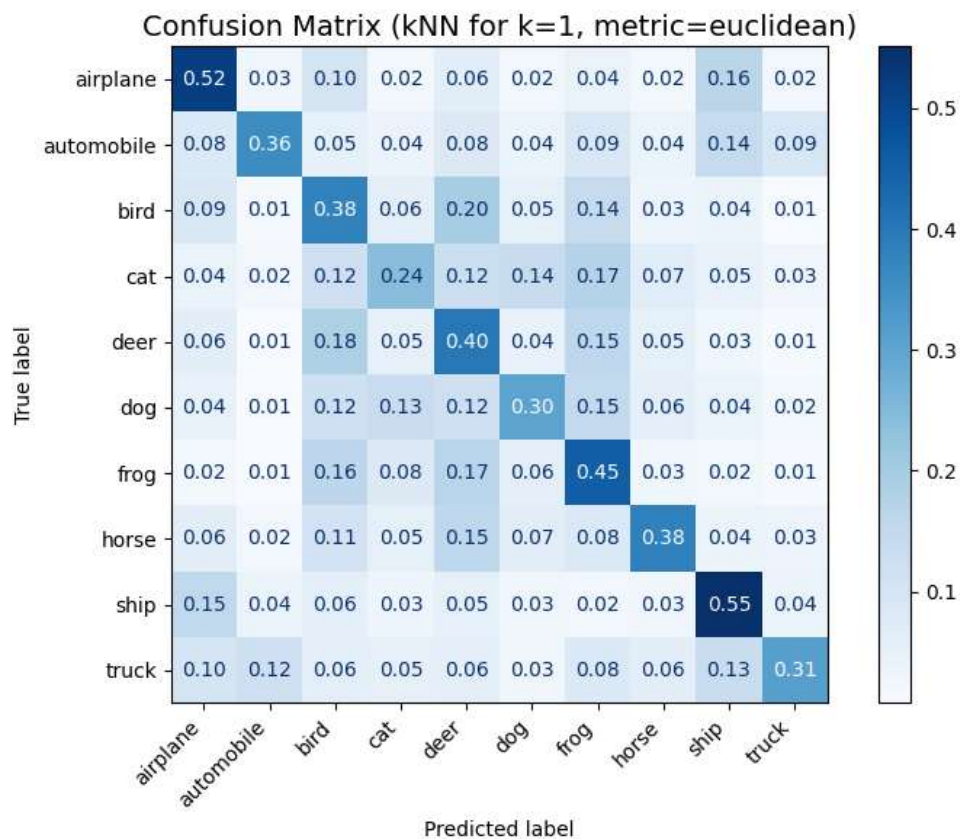
Η Ευκλείδεια απόσταση μετράει την ευθεία γραμμή (shortest path) μεταξύ δύο σημείων σε έναν χώρο και είναι η πιο διαδεδομένη μετρική για το kNN. Ο τύπος της είναι: $d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$

Testing time: 8.90 seconds

Test accuracy: 38.93%

Class	Precision	Recall	F1-score	Support
Airplane	0.44	0.52	0.48	1000
Automobile	0.56	0.36	0.44	1000
Bird	0.28	0.38	0.32	1000
Cat	0.32	0.24	0.27	1000
Deer	0.28	0.40	0.33	1000
Dog	0.39	0.30	0.34	1000
Frog	0.33	0.45	0.38	1000
Horse	0.50	0.38	0.43	1000
Ship	0.46	0.55	0.50	1000
Truck	0.54	0.31	0.40	1000
Accuracy			0.39	10000
Macro Avg	0.41	0.39	0.39	10000
Weighted Avg	0.41	0.39	0.39	10000

Table 1: Classification Report (kNN for k=1, metric=euclidean)



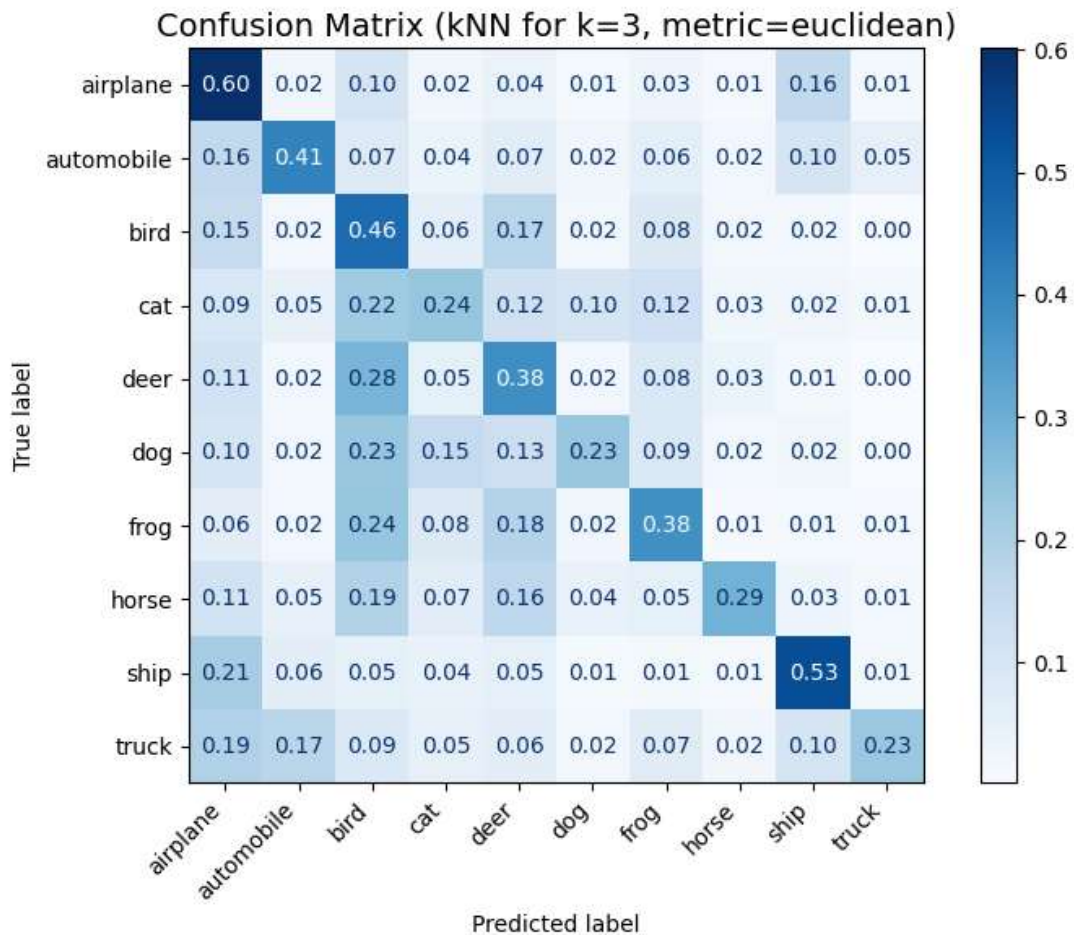
➤ **Euclidean distance, k = 3:**

Testing time: 8.57 seconds

Test accuracy: 37.52%

Class	Precision	Recall	F1-score	Support
Airplane	0.34	0.60	0.43	1000
Automobile	0.49	0.41	0.45	1000
Bird	0.24	0.46	0.31	1000
Cat	0.30	0.24	0.26	1000
Deer	0.28	0.38	0.32	1000
Dog	0.47	0.23	0.31	1000
Frog	0.39	0.38	0.38	1000
Horse	0.65	0.29	0.40	1000
Ship	0.53	0.53	0.53	1000
Truck	0.66	0.23	0.34	1000
Accuracy			0.38	10000
Macro Avg	0.43	0.38	0.37	10000
Weighted Avg	0.43	0.38	0.37	10000

Table 2: Classification Report (kNN for k=3, metric=euclidean)



➤ **Cosine distance, k = 1:**

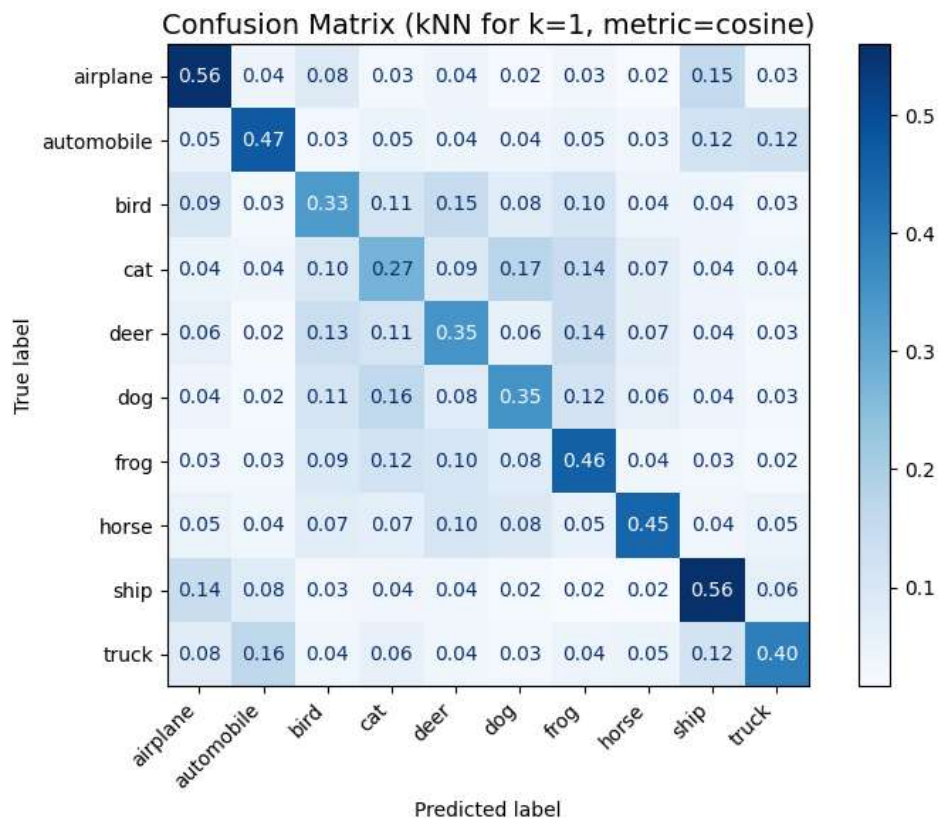
Η Cosine similarity/distance μετράει την ομοιότητα μεταξύ δύο διανυσμάτων υπολογίζοντας το συνημίτονο της γωνίας μεταξύ τους. Ο τύπος είναι: $similarity = \cos \theta = \frac{A \cdot B}{\|A\| \cdot \|B\|}$

Testing time: 34.86 seconds

Test accuracy: 41.94%

Class	Precision	Recall	F1-score	Support
Airplane	0.49	0.56	0.52	1000
Automobile	0.51	0.47	0.49	1000
Bird	0.33	0.33	0.33	1000
Cat	0.27	0.27	0.27	1000
Deer	0.34	0.35	0.34	1000
Dog	0.38	0.35	0.36	1000
Frog	0.40	0.46	0.43	1000
Horse	0.53	0.45	0.49	1000
Ship	0.48	0.56	0.51	1000
Truck	0.49	0.40	0.44	1000
Accuracy			0.42	10000
Macro Avg	0.42	0.42	0.42	10000
Weighted Avg	0.42	0.42	0.42	10000

Table 3: Classification Report (kNN for k=1, metric=cosine)



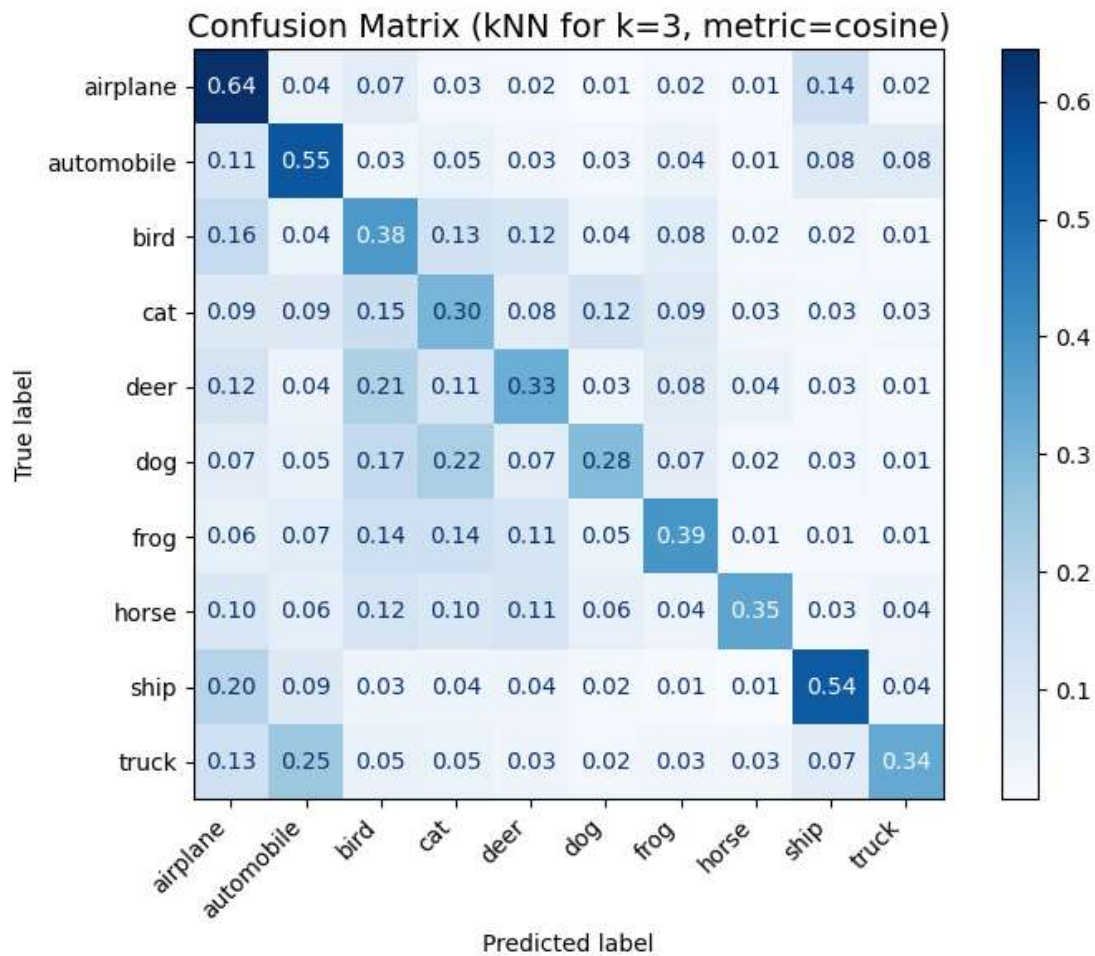
➤ **Cosine distance, k = 3:**

Testing time: 33.51 seconds

Test accuracy: 41.07%

Class	Precision	Recall	F1-score	Support
Airplane	0.38	0.64	0.48	1000
Automobile	0.43	0.55	0.48	1000
Bird	0.28	0.38	0.32	1000
Cat	0.26	0.30	0.28	1000
Deer	0.35	0.33	0.34	1000
Dog	0.43	0.28	0.34	1000
Frog	0.47	0.39	0.43	1000
Horse	0.66	0.35	0.46	1000
Ship	0.55	0.54	0.55	1000
Truck	0.58	0.34	0.43	1000
Accuracy			0.41	10000
Macro Avg	0.44	0.41	0.41	10000
Weighted Avg	0.44	0.41	0.41	10000

Table 4: Classification Report (kNN for k=3, metric=cosine)



➤ **Manhattan distance, k = 1:**

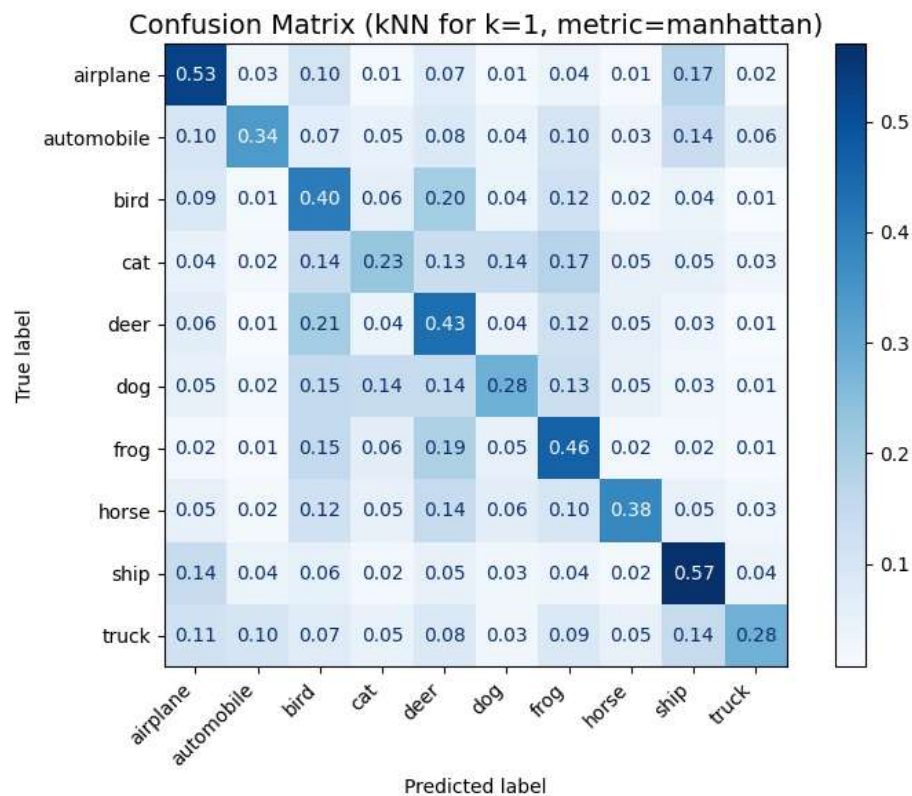
Η απόσταση Manhattan, γνωστή και ως "taxicab distance", υπολογίζει το άθροισμα των απόλυτων διαφορών των συντεταγμένων, δηλαδή την απόσταση που θα διανύαμε αν μπορούσαμε να κινηθούμε μόνο οριζοντίως ή καθέτως. Ο τύπος της είναι: $d(x, y) = \sum_{i=1}^n |x_i - y_i|$

Testing time: 51.92 seconds

Test accuracy: 39.08%

Class	Precision	Recall	F1-score	Support
Airplane	0.44	0.53	0.48	1000
Automobile	0.57	0.34	0.42	1000
Bird	0.27	0.40	0.33	1000
Cat	0.32	0.23	0.27	1000
Deer	0.28	0.43	0.34	1000
Dog	0.39	0.28	0.33	1000
Frog	0.34	0.46	0.39	1000
Horse	0.55	0.38	0.45	1000
Ship	0.46	0.57	0.51	1000
Truck	0.57	0.28	0.37	1000
Accuracy			0.39	10000
Macro Avg	0.42	0.39	0.39	10000
Weighted Avg	0.42	0.39	0.39	10000

Table 5: Classification Report (kNN for k=1, metric=manhattan)



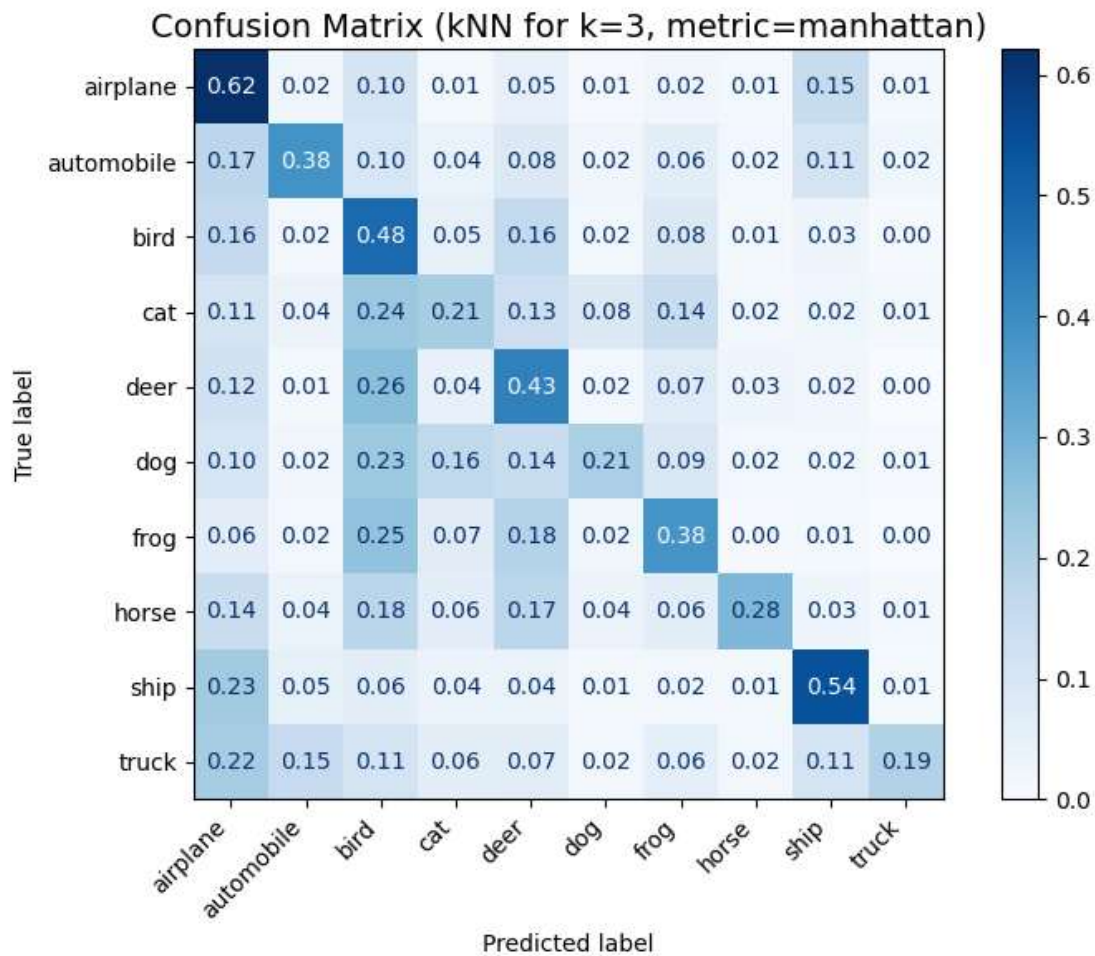
➤ **Manhattan distance, k = 3:**

Testing time: 40.20 seconds

Test accuracy: 37.23%

Class	Precision	Recall	F1-score	Support
Airplane	0.32	0.62	0.43	1000
Automobile	0.51	0.38	0.44	1000
Bird	0.24	0.48	0.32	1000
Cat	0.29	0.21	0.24	1000
Deer	0.29	0.43	0.35	1000
Dog	0.47	0.21	0.29	1000
Frog	0.38	0.38	0.38	1000
Horse	0.68	0.28	0.40	1000
Ship	0.53	0.54	0.53	1000
Truck	0.73	0.19	0.30	1000
Accuracy			0.37	10000
Macro Avg	0.44	0.37	0.37	10000
Weighted Avg	0.44	0.37	0.37	10000

Table 6: Classification Report (kNN for k=3, metric=manhattan)



2.2 Nearest Class Centroid

Ο αλγόριθμος Nearest Class Centroid βασίζεται στην ιδέα ότι κάθε κλάση αναπαρίσταται με ένα ενιαίο σημείο, το κέντρο (centroid) των δεδομένων της.

- 1. Υπολογισμός κέντρων:** για κάθε κλάση υπολογίζεται ο μέσος όρος (κέντρο) των χαρακτηριστικών της, ο οποίος χαρακτηρίζεται ως ένα νέο σημείο: $c_k = \frac{1}{n_k} \sum_{j=1}^{n_k} x_j$
- 2. Υπολογισμός αποστάσεων:** για κάθε νέο δείγμα υπολογίζεται η απόστασή του από όλα τα υπολογισμένα κέντρα. Παρακάτω εξετάστηκαν διάφοροι τύποι υπολογισμού αυτής της απόστασης.
- 3. Κατηγοριοποίηση:** εύρεση της μικρότερης απόστασης, δηλαδή του πλησιέστερου centroid και ένταξη του δείγματος στην αντίστοιχη κλάση.

Η μέθοδος αυτή συνεπώς κάνει preprocessing και εκπαιδεύεται μόνο στα κέντρα των δεδομένων, τα οποία και αποθηκεύει. Αναμένουμε λοιπόν ταχύτερα αποτελέσματα (σε σχέση με το kNN) σε μεγάλα datasets, καθώς η πολυπλοκότητα εξαρτάται από τον αριθμό κλάσεων, όχι από τον αριθμό των δειγμάτων. Ωστόσο, η απλοϊκότητά του μπορεί να οδηγήσει σε χειρότερη ακρίβεια για πολύπλοκα προβλήματα.

Το μοντέλο αυτό, όπως και το kNN, χτίστηκε πάνω στο *training set*, εξετάστηκε τόσο στο *training* όσο και στο *test set*, αλλά δόθηκε βάση στην μελέτη των *test set* αποτελεσμάτων. Η υλοποίησή του βρίσκεται στο αρχείο *nearestClassCentroid.py*.

➤ Euclidean distance

Η Ευκλείδεια απόσταση υπολογίζει την ευθεία γραμμή (shortest path) από το test point στο centroid κάθε κλάσης. Ο τύπος της είναι:

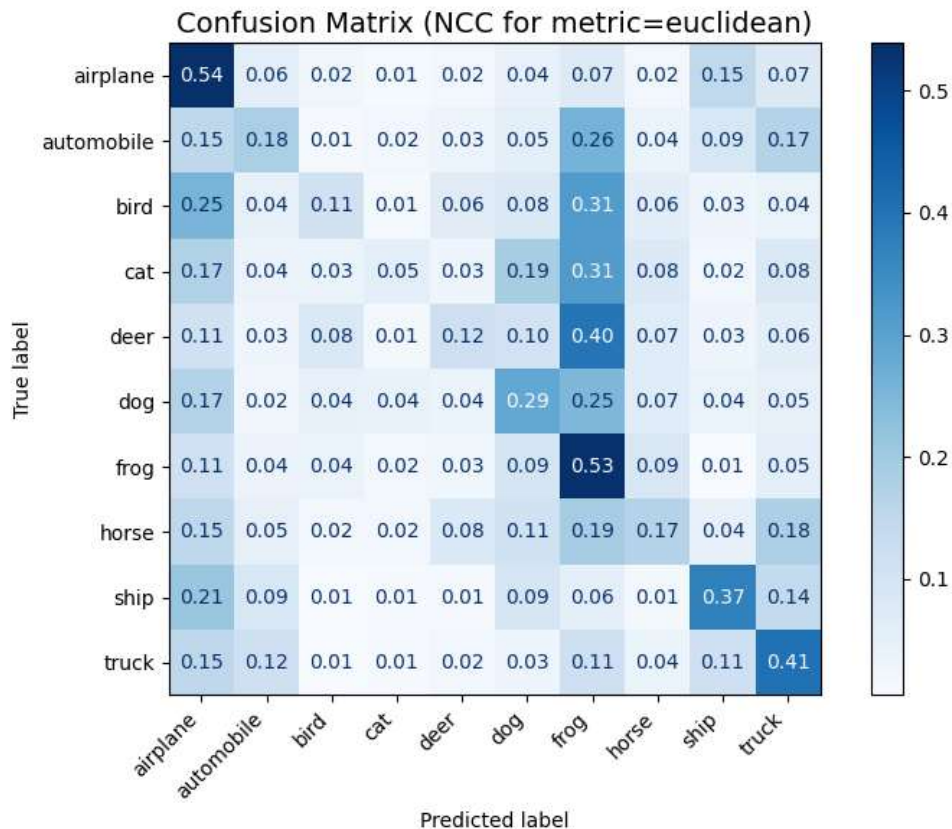
$$d(x, c_k) = \sqrt{\sum_{i=1}^n (x_i - c_{ki})^2}$$

Testing time: 0.03 seconds

Test accuracy: 27.60%

Class	Precision	Recall	F1-score	Support
Airplane	0.27	0.54	0.36	1000
Automobile	0.28	0.18	0.22	1000
Bird	0.29	0.11	0.16	1000
Cat	0.26	0.05	0.09	1000
Deer	0.27	0.12	0.16	1000
Dog	0.27	0.29	0.28	1000
Frog	0.21	0.53	0.31	1000
Horse	0.26	0.17	0.20	1000
Ship	0.42	0.37	0.39	1000
Truck	0.33	0.41	0.36	1000
Accuracy			0.28	10000
Macro Avg	0.29	0.28	0.25	10000
Weighted Avg	0.29	0.28	0.25	10000

Table 7: Classification Report (NCC for metric=euclidean)



➤ Manhattan distance

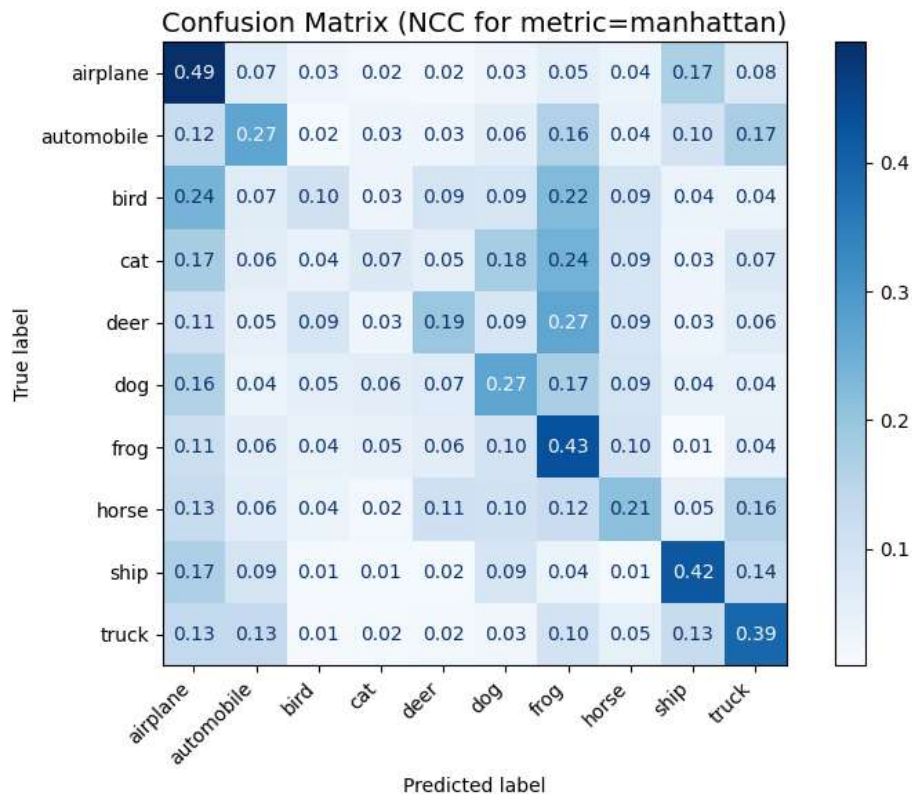
Η απόσταση Manhattan, γνωστή και ως "taxicab distance", υπολογίζει την απόσταση "grid-based" από το test point στο centroid κάθε κλάσης, δηλαδή την απόσταση που θα διανύσαμε αν μπορούσαμε να κινηθούμε μόνο οριζοντίως ή καθέτως. Ο τύπος της είναι: $d(x, c_k) = \sum_{i=1}^n |x_i - c_{ki}|$

Testing time: 0.01 seconds

Test accuracy: 28.52%

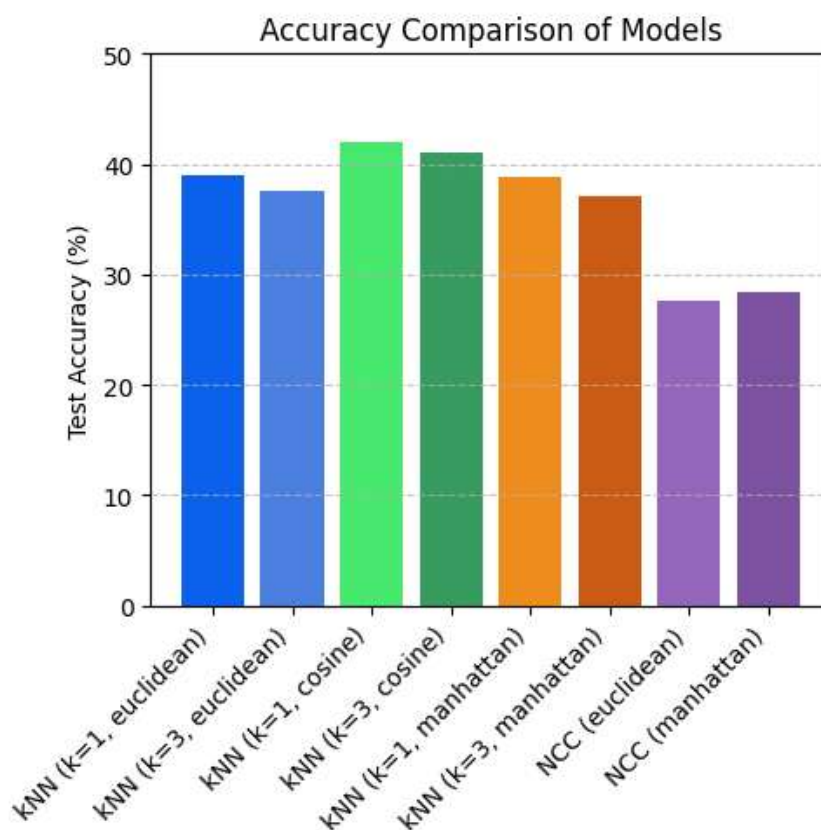
Class	Precision	Recall	F1-score	Support
Airplane	0.27	0.49	0.35	1000
Automobile	0.30	0.27	0.28	1000
Bird	0.23	0.10	0.14	1000
Cat	0.21	0.07	0.10	1000
Deer	0.30	0.19	0.23	1000
Dog	0.27	0.27	0.27	1000
Frog	0.24	0.43	0.31	1000
Horse	0.26	0.21	0.23	1000
Ship	0.42	0.42	0.42	1000
Truck	0.33	0.39	0.36	1000
Accuracy			0.29	10000
Macro Avg	0.28	0.29	0.27	10000
Weighted Avg	0.28	0.29	0.27	10000

Table 8: Classification Report (NCC for metric=manhattan)



2.3 Σύγκριση – Συμπεράσματα

Όπως φαίνεται, η μετρική *Cosine* υπερτερεί σταθερά για τον kNN, με τη βέλτιστη απόδοση να επιτυγχάνεται για $k=3$ (43.51%). Η αύξηση του k από 1 σε 3 φαίνεται να βελτιώνει την ακρίβεια σε όλες τις μετρικές. Αντιθέτως, ο αλγόριθμος NCC παρουσίασε χαμηλότερη απόδοση σε σύγκριση με τον kNN, πιθανώς λόγω της απλουστευμένης υπόθεσης ότι κάθε κλάση μπορεί να αναπαρασταθεί από ένα μόνο κέντρο. Η μετρική *Cosine* φαίνεται να είναι πιο κατάλληλη για τα χαρακτηριστικά που προκύπτουν από το PCA, ενώ η *Manhattan* και *Euclidean* έχουν παρόμοια αλλά χειρότερη απόδοση. Γενικότερα, συμπεραίνουμε ότι για οποιαδήποτε από τις παραπάνω μεθόδους ή μετρικές, το accuracy κυμαίνεται στο 30-40%, κάτι το οποίο ήταν και αναμενόμενο για ένα dataset τόσο μεγάλου εύρους. Τα παραπάνω αποτελέσματα βρίσκονται στο αρχείο *main.ipynb*.



3 Μέρος B – Convolutional Neural Network

Στο κύριο μέρος της εργασίας που ακολουθεί, υλοποιήθηκε ένα Συνελικτικό Νευρωνικό Δίκτυο (CNN), το οποίο κρίθηκε καταλληλότερο για ένα database όπως το CIFAR-10 σε σύγκριση με ένα Multilayer Perceptron (MLP). Ένα MLP «βλέπει» την εικόνα σαν απλό διάνυσμα 3072 τιμών ($32 \times 32 \times 3$) χωρίς να αναγνωρίζει τη χωρική συσχέτιση γειτονικών pixels. Αντιθέτως, ένα CNN χρησιμοποιεί φίλτρα που ανιχνεύουν άκρα, υφές και σχήματα σε όλες τις θέσεις της εικόνας, αποκτώντας μεταθετική αμεταβλητότητα (αν μετακινηθεί λίγο το αντικείμενο, το pattern αναγνωρίζεται πάλι).

Αρχικά γίνεται παρόμοια προεπεξεργασία του dataset: εισαγωγή του dataset μέσω της βιβλιοθήκης Keras της Tensorflow και κανονικοποίηση με εύρος $[0, 255] \rightarrow [0, 1]$. Σε αυτή τη φάση απαραίτητη είναι η διατήρηση των αρχικών διαστάσεων των εικόνων για ορθή εισαγωγή τους στο πρώτο επίπεδο του μοντέλου:

- Training data shape: (50000, 32, 32, 3)
- Test data shape: (10000, 32, 32, 3)

3.1 Architecture and Hyperparameters

Conv2D Layers: Τα layers Conv2D με φίλτρα 32, 64, 128 και 256 καναλιών εξάγουν τοπικά χωρικά χαρακτηριστικά από τις εικόνες, εφαρμόζοντας συνελίξεις 3×3 με padding 'same' (zero padding το οποίο διατηρεί τις αρχικές διαστάσεις του input) και συνάρτηση ενεργοποίησης ReLU (Rectified Liner Unit), επιτρέποντας στο δίκτυο να μάθει ακμές, υφές και πιο σύνθετα μοτίβα σε διαδοχικά βάρη. Η συνάρτηση ReLU επιτρέπει την έλευση όλων των θετικών τιμών και τον μηδενισμό όλων των αρνητικών, βοηθώντας στη διατήρηση της μη γραμμικότητας χωρίς περίπλοκους μετασχηματισμούς, επιτρέποντας στα μοντέλα να μαθαίνουν πιο αποτελεσματικά.

Σε όλα τα μοντέλα οι συνελίξεις οργανώνονται σε blocks (π.χ. Block 1 με 32 φίλτρα, Block 2 με 64 κλπ.), ώστε στα πρώτα επίπεδα να αποδομούνται απλά χαρακτηριστικά και στα βαθύτερα να συλλαμβάνονται πιο αφηρημένες αναπαραστάσεις αντικειμένων.

MaxPooling2D Layers: Τα MaxPooling2D layers μειώνουν τις χωρικές διαστάσεις, κρατώντας το μέγιστο σε κάθε γειτονιά 2×2 , γεγονός που μειώνει το πλήθος των παραμέτρων και εισάγει μεταφορική αμεταβλητότητα στα τοπικά χαρακτηριστικά. Μετά από κάθε block συνελίξεων υποδιπλασιάζει τις διαστάσεις του feature map, επιτρέποντας

πιο βαθιά δίκτυα με αποδεκτή υπολογιστική πολυπλοκότητα και συμβάλλοντας στο φιλτράρισμα θορύβου.

Batch Normalization Layers: Καθώς τα weights των προηγούμενων layers αλλάζουν, η κατανομή των τιμών που φτάνουν σε ένα layer (mean, variance) μετακινείται, και το layer πρέπει συνεχώς να «ξαναπροσαρμόζεται». Το Batch Normalization αποτελεί ένα επιπλέον layer που «κανονικοποιεί», δηλαδή επαναφέρει αυτές τις τιμές σε μια πιο σταθερή κλίμακα (zero-mean, unit-variance) μέσα σε κάθε mini-batch, κάνοντας το training πιο γρήγορο και πιο σταθερό.

Dropout Layers: Τα Dropout layers απενεργοποιούν τυχαίο ποσοστό p των νευρώνων σε κάθε βήμα εκπαίδευσης, περιορίζοντας την εξάρτηση του μοντέλου από αυτούς και μειώνοντας το overfitting. Στα πειράματα χρησιμοποιήθηκαν διαφορετικά ποσοστά dropout ανά block (π.χ. 0.2, 0.3, 0.4 στα συνελκτικά blocks και 0.3, 0.5 στο τελικό Dense), ώστε να συγκριθεί η ισορροπία μεταξύ γενίκευσης και ικανότητας εκμάθησης πολύπλοκων μοτίβων.

Flatten και Dense Layers: Το Flatten() μετατρέπει τα 3D feature maps σε μονοδιάστατο διάνυσμα, το οποίο τροφοδοτείται σε πλήρως συνδεδεμένα (Dense) layers, όπου πραγματοποιείται η τελική μη γραμμική ταξινόμηση με ReLU στο κρυφό layer και με softmax στην έξοδο.

Η έξοδος του δικτύου σε όλα τα μοντέλα είναι ένα Dense layer 10 νευρώνων με συνάρτηση ενεργοποίησης softmax, ώστε να μοντελοποιείται απευθείας η κατανομή πιθανότητας στις 10 κλάσεις του CIFAR-10:

$$\sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}.$$

GlobalAveragePooling2D Layers: Στα τελευταία μοντέλα εισάγεται GlobalAveragePooling2D, το οποίο αντικαθιστά το Flatten συμπτύσσοντας κάθε feature map σε έναν μέσο όρο, μειώνοντας δραστικά τον αριθμό παραμέτρων και κάνοντας το μοντέλο λιγότερο επιρρεπές σε overfitting.

“Adam” Optimizer: Σε όλα τα μοντέλα ως optimizer χρησιμοποιείται ο Adam (Adaptive Moment Estimation), μέθοδος η οποία, προσαρμόζει τον ρυθμό εκμάθησης ανά παράμετρο. Διατηρήθηκε σταθερός σε όλα τα μοντέλα, καθώς σε σύγκριση με άλλους βελτιστοποιητές τους ξεπερνά σημαντικά τόσο όσον αφορά τον χρόνο εκπαίδευσης όσο και την ακρίβεια σύγκλισης, καθιστώντας τον ιδιαίτερα χρήσιμο σε σύνθετα μοντέλα με μεγάλα σύνολα δεδομένων.

Sparse Categorical Cross-entropy Loss: Για τα πρώτα επτά μοντέλα χρησιμοποιήθηκε $loss = 'sparse_categorical_crossentropy'$, όπου

οι ετικέτες είναι ακέραιοι $\{0, \dots, 9\}$ και η συνάρτηση απώλειας εφαρμόζει αυτόματα one-hot κωδικοποίηση εσωτερικά, υπολογίζοντας το cross-entropy loss, δηλαδή τη διαφορά μεταξύ της πραγματικής κατανομής πιθανοτήτων και της προβλεπόμενης κατανομής πιθανοτήτων. Η χρήση της sparse έκδοσης επιτρέπει να κρατηθούν τα labels στη φυσική μορφή που παρέχει το CIFAR-10 αποφεύγοντας την εφαρμογή one-hot encoding. Η αποφυγή αυτή έγινε καθώς δεν υπάρχουν εικόνες που να ανήκουν ταυτοχρόνως σε παραπάνω από 1 κλάση, οπότε μια one-hot υλοποίηση θα οδηγούσε σε άσκοπη αύξηση υπολογιστικού κόστους.

Custom Cross-entropy loss με Label Smoothing: Στο 9ο μοντέλο υλοποιείται η custom συνάρτηση *sparse_cce_label_smoothing* (Sparse Categorical Crossentropy Loss with Label smoothing), η οποία μετατρέπει εσωτερικά τα ακέραια labels σε one-hot και εφαρμόζει categorical cross-entropy με label smoothing $\alpha = 0.1$, «μαλακώνοντας» τους στόχους πιθανότητας ώστε η σωστή κλάση να μην έχει ακριβώς πιθανότητα 1.

Batch size: Σε όλα τα μοντέλα επιλέχθηκε batch size 64, ένας πολύ συνηθισμένος, πρακτικός συμβιβασμός ανάμεσα σε σταθερά gradients, καλή γενίκευση και αποδοτική χρήση της GPU. Τιμές όπως 64 ή 128 δίνουν αρκετά δείγματα ανά update ώστε τα gradients να είναι σταθερά, αλλά όχι τόσο πολλά ώστε το δίκτυο να «βολεύεται» υπερβολικά στα στατιστικά του training set και να χάνει σε test accuracy.

Validation Set: Σε όλα τα μοντέλα η εκπαίδευση γίνεται με *validation_split = 0.2* πάνω στο *x_train*, *y_train* (δηλαδή από τα 50.000 δείγματα του training set, διατηρούνται τα 40.000 για training και τα 10.000 για validation). Στα μοντέλα με Data Augmentation όπου απαιτείται χειροκίνητος διαχωρισμός του training/validation set, χρησιμοποιείται η παράμετρος *stratify = y_train* ώστε να εξασφαλίζεται ισορροπία κλάσεων στο validation. Η διαδικασία αυτή εξασφαλίζει τη διατήρηση του test set αποκλειστικά για testing, ώστε να εξάγονται όσο το δυνατόν πιο ρεαλιστικά αποτελέσματα.

Epochs: Ο αριθμός των εποχών αυξάνεται σταδιακά (10, 20, 30, 50, 80) και στα τελευταία μοντέλα συνδυάζεται με Early Stopping και ModelCheckpoint, ώστε να σταματά η εκπαίδευση όταν το validation loss παύει να βελτιώνεται, ενώ παράλληλα αποθηκεύεται το καλύτερο μοντέλο.

Data Augmentation: Για κάποια από τα μοντέλα χρησιμοποιείται ImageDataGenerator, που εφαρμόζει τυχαίες περιστροφές έως 15° , οριζόντιες μετατοπίσεις 10%, κάθετες μετατοπίσεις 10%, οριζόντια αναστροφή και zoom $\pm 10\%$, αυξάνοντας το ποικιλόμορφο training set.

Early Stopping: Πρόκειται για μια τεχνική regularization που σταματά την εκπαίδευση του νευρωνικού δικτύου πριν εξαντληθεί ο μέγιστος αριθμός εποχών, όταν διαπιστωθεί ότι το μοντέλο αρχίζει να υπερπροσαρμόζεται στα δεδομένα εκπαίδευσης. Παρακολουθείται συνήθως η τιμή της απώλειας (ή της ακρίβειας) στο σύνολο επικύρωσης και όταν αυτή πάψει να βελτιώνεται για έναν συγκεκριμένο αριθμό εποχών, η εκπαίδευση τερματίζεται πρόωρα.

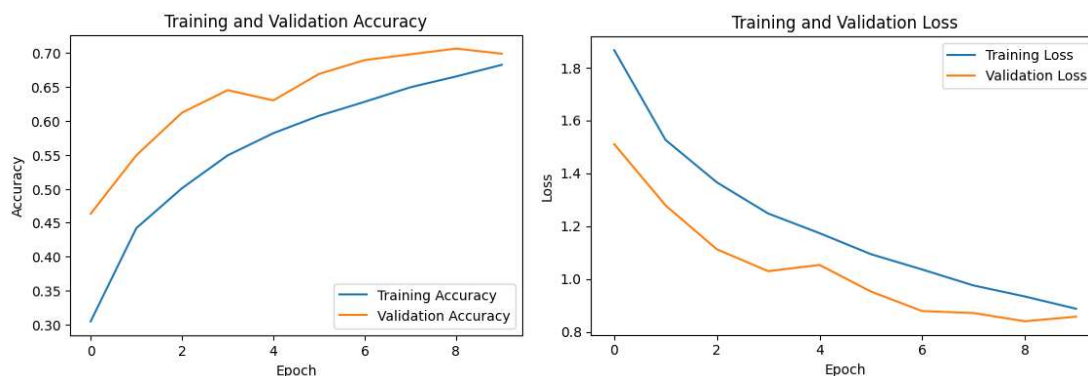
ReduceLROnPlateau: Η τεχνική ReduceLROnPlateau είναι μια δυναμική στρατηγική προσαρμογής του ρυθμού μάθησης (learning rate), η οποία μειώνει αυτόματα το learning rate όταν η επίδοση στο validation set πάψει να βελτιώνεται. Σε αντίθεση με ένα σταθερό ή χειροκίνητα προγραμματισμένο learning rate, το ReduceLROnPlateau αντιδρά στην πραγματική συμπεριφορά της καμπύλης validation loss, μειώνοντας τον ρυθμό μάθησης «όταν χρειάζεται».

3.2 Model Training

Η αναλυτική υλοποίηση των μοντέλων βρίσκεται στο αρχείο CNN.ipynb, ενώ βοηθητικές συναρτήσεις για εκπαίδευση και εξαγωγή αποτελεσμάτων βρίσκονται στο αρχείο convolutionalNeuralNetwork.py.

Model 1: Στην υλοποίηση του base model χρησιμοποιούνται λίγα layers και μικρά φίλτρα, ενώ η εκπαίδευση πραγματοποιείται για batch size = 64 και διαρκεί μόνο για 10 εποχές. Αυτή η απλή αρχιτεκτονική μας δίνει μια γρήγορη πρώτη εικόνα για την απόδοση του δικτύου. Τα αποτελέσματα που προκύπτουν είναι τα εξής:

- Training time: 118.45 seconds
- Test accuracy: 69.87%
- Test loss: 0.8736

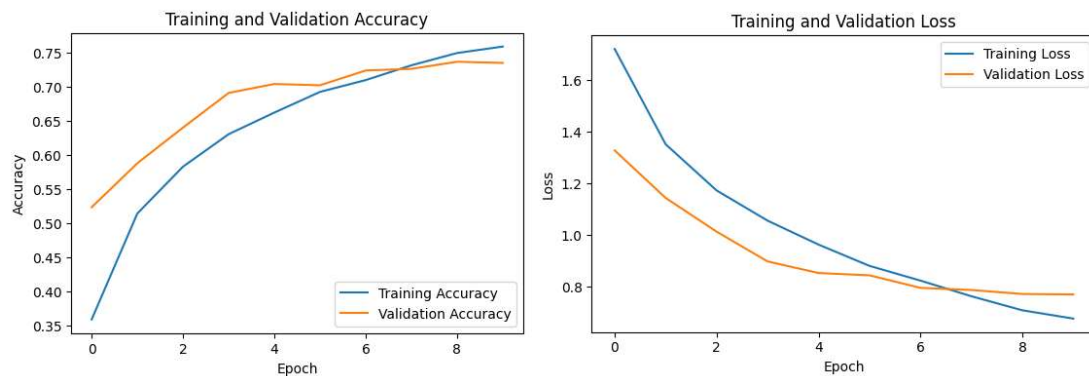


Από τις καμπύλες εκμάθησης παρατηρείται ελαφρώς το φαινόμενο underfitting, καθώς τόσο η καμπύλη training loss, όσο και η training accuracy δεν φαίνεται να φτάνουν σε κορεσμό. Η validation loss

βρίσκεται συνεχώς υπό της training loss γεγονός που δείχνει την ελλιπή προσαρμογή.

Model 2: Ακολουθείται παρόμοια αρχιτεκτονική με ελαφρώς πιο εμπλουτισμένα layers, αλλά όμοιες υπερπαραμέτρους και πλήθος εποχών εκπαίδευσης. Τα αποτελέσματα που προκύπτουν είναι τα εξής:

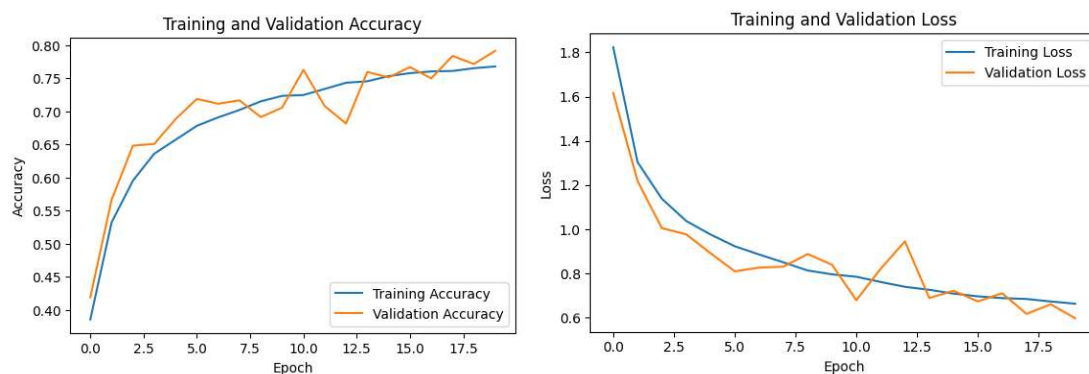
- Training time: 187.20 seconds
- Test accuracy: 73.01%
- Test loss: 0.7975



Με τον εμπλουτισμό των layers παρατηρείται αύξηση του test accuracy, ωστόσο οι καμπύλες training accuracy και training loss εξακολουθούν να μην συγκλίνουν πλήρως, γεγονός που υποδεικνύει τη δυνατότητα περαιτέρω εκμάθησης. Άρα συμπεραίνουμε ότι απαραίτητη στο επόμενο βήμα είναι η αύξηση των epochs.

Model 3: Γίνεται αύξηση των εποχών εκπαίδευσης από 10 σε 20 όπως διαπιστώθηκε ότι χρειάζεται, αλλά για διασφάλιση γρήγορου training και αποφυγή overfitting προστίθενται Batch Normalization και Dropout Layers. Τα αποτελέσματα που προκύπτουν είναι τα εξής:

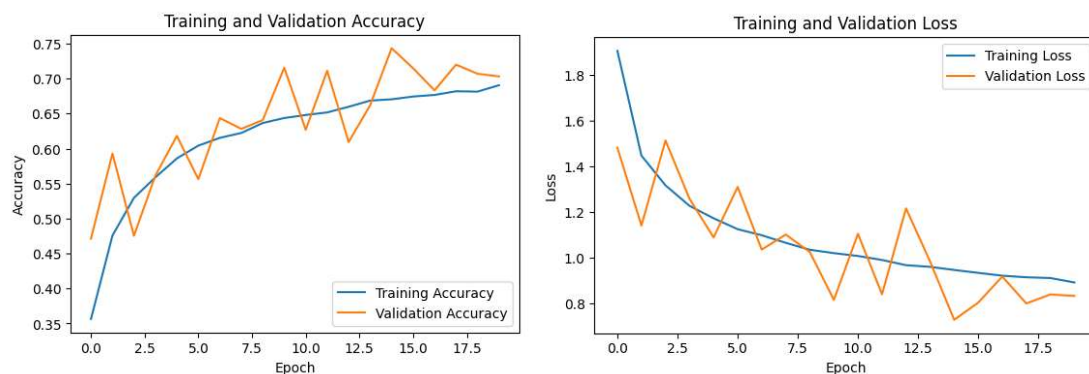
- Training time: 650.98 seconds
- Test accuracy: 78.90%
- Test loss: 0.6108



Αυτή τη φορά παρατηρείται σημαντική αύξηση του test accuracy και μείωση του test loss. Οι καμπύλες training accuracy και training loss πλησιάζουν όλο και περισσότερο στη σύγκλιση, ενώ οι validation accuracy και validation loss τις ακολουθούν πιστά με μικρές ταλαντώσεις.

Model 4: Διατηρώντας ακριβώς ίδια αρχιτεκτονική και πλήθος εποχών προστίθεται Data Augmentation. Τα αποτελέσματα που προκύπτουν είναι τα εξής:

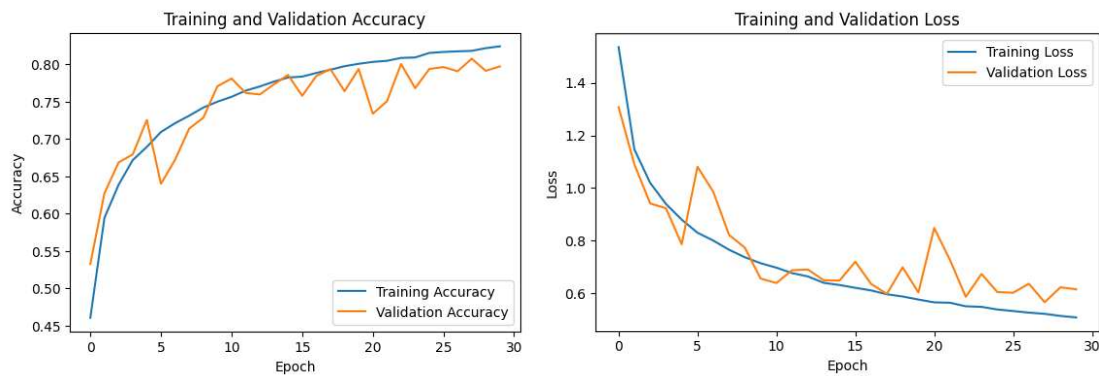
- Training time: 762.10 seconds
- Test accuracy: 70.18%
- Test loss: 0.8445



Παρατηρούμε ότι ο συνδυασμός Batch Normalization, πολλαπλά Dropout layers και Data Augmentation μειώνει σημαντικά την απόδοση του test set τόσο σε accuracy όσο και σε loss. Οι καμπύλες validation accuracy και validation loss παρουσιάζουν ακόμα περισσότερες ταλαντώσεις, το οποίο υποδεικνύει περισσότερο θόρυβο, μεγαλύτερη στοχαστικότητα και υπερβολικό regularization.

Model 5: Πραγματοποιείται μείωση των Dropout Layers (διατηρείται μόνο 1 στην έξοδο του μοντέλου ανάμεσα στα Dense Layers), καθώς και αύξηση του αριθμού εποχών σε 30. Τα αποτελέσματα που προκύπτουν είναι τα εξής:

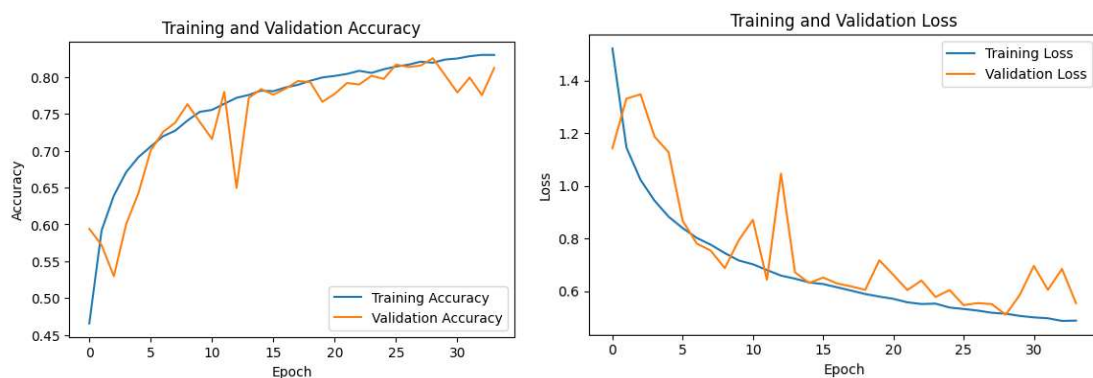
- Training time: 1120.75 seconds
- Test accuracy: 78.92%
- Test loss: 0.6420



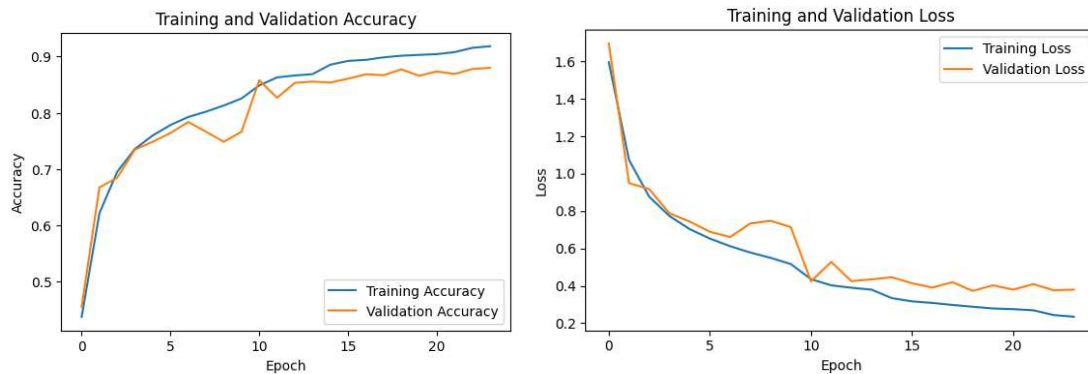
Παρατηρείται σαφώς καλύτερη απόδοση του test set, με ελαφριά μείωση των ταλαντώσεων σε validation accuracy και loss, αλλά όχι πλήρη περιορισμό αυτών. Συνεπώς, ελοχεύει μικρός κίνδυνος overfitting παρόλο που οι καμπύλες training accuracy/loss δεν έχουν φτάσει σε πλήρη κορεσμό.

Model 6: Στη φάση αυτή στοχεύοντας στην παρατήρηση συμπεριφοράς του μοντέλου σε πλήρη κορεσμό, δεν γίνεται ακόμα τροποποίηση της αρχιτεκτονικής του αλλά αυξάνονται οι εποχές σε 50. Γίνεται χρήση της μεθόδου Early Stopping για αποφυγή περιπτώσεων βημάτων εκπαίδευσης. Τα αποτελέσματα που προκύπτουν είναι τα εξής:

- Training time: 1557.82 seconds
- Test accuracy: 82.50%
- Test loss: 0.5143
- Model stopped at epoch 34, restoring weights from epoch 29



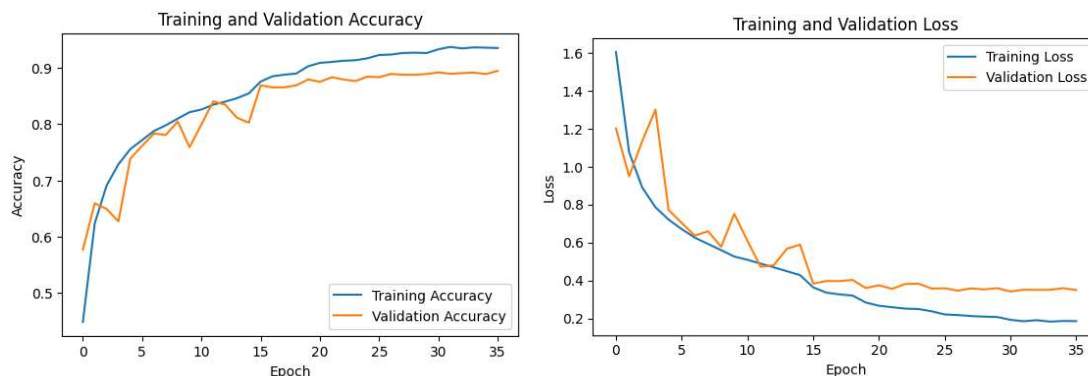
Η συμπεριφορά του test set βελτιώνεται όλο και περισσότερο με accuracy που ξεπερνάει το 80% και loss κοντά στο 0.5. Οι καμπύλες training accuracy/loss ακολουθούν μια ομαλή πορεία προς σύγκλιση, ενώ οι validation accuracy/loss κινούνται πιστά μεν γύρω από το training, με διαρκή ταλάντωση δε από την αρχή έως το τέλος. Με Data Augmentation και Dropout, ο θόρυβος στο training είναι αυξημένος, οπότε μικρές διαφορές στα weights από εποχή σε εποχή μεταφράζονται σε σχετικά μεγάλες διαφορές σε validation loss/accuracy. Άρα οι μεταπτώσεις γύρω



Παρατηρείται σημαντική αύξηση του test accuracy και μείωση του test loss, το οποίο σε συνδυασμό με τις καμπύλες εκμάθησης προδίδουν ένα δυνατό μοντέλο με καλή γενίκευση και πολύ ήπιο overfitting. Οι καμπύλες validation accuracy/loss ακολουθούν με ελαφρές ταλαντώσεις τις καμπύλες training accuracy/loss καταλήγοντας σε ένα μικρό gap, το οποίο όμως παραμένει σταθερό αποτρέποντας το μη αποδεκτό overfitting.

Model 9: Για την αποτροπή του overfitting αυξάνεται το Dropout και επιστρέφουμε στις 50 εποχές εκπαίδευσης. Τα αποτελέσματα που προκύπτουν είναι τα εξής:

- Training time: 4394.93 seconds
- Test accuracy: 88.64%
- Test loss: 0.3629
- Model stopped at epoch 36, restoring weights from epoch 31

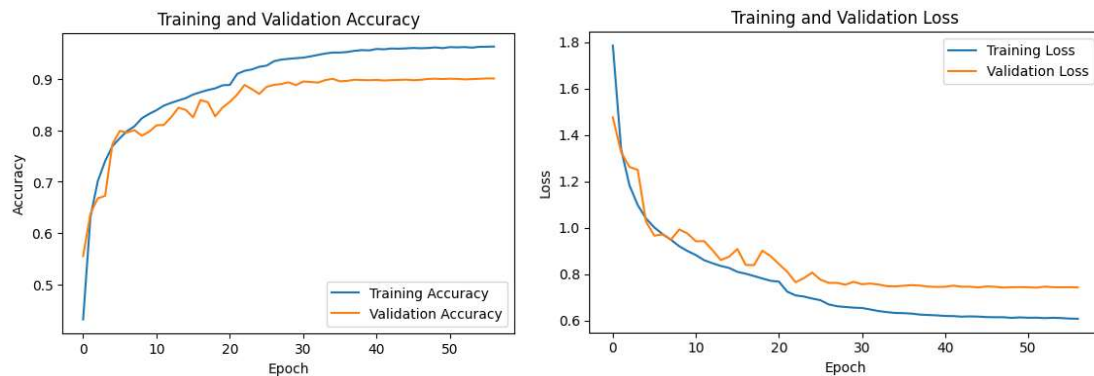


Παρατηρούμε ότι παρά την καλύτερη απόδοση του test set, οι καμπύλες εκμάθησης φανερώνουν υπερβολικό regularization. Οι ταλαντώσεις είναι πιο έντονες και αντιλαμβανόμαστε ότι η αύξηση του Dropout πιθανώς κόβει χρήσιμη ικανότητα μάθησης, χωρίς να χαρίζει ουσιαστικό κέρδος στη γενίκευση.

Model 10: Σε αυτή τη φάση επιστρέφουμε στην αρχιτεκτονική του 8ου μοντέλου και γίνεται δοκιμή χρήσης μιας custom συνάρτησης για

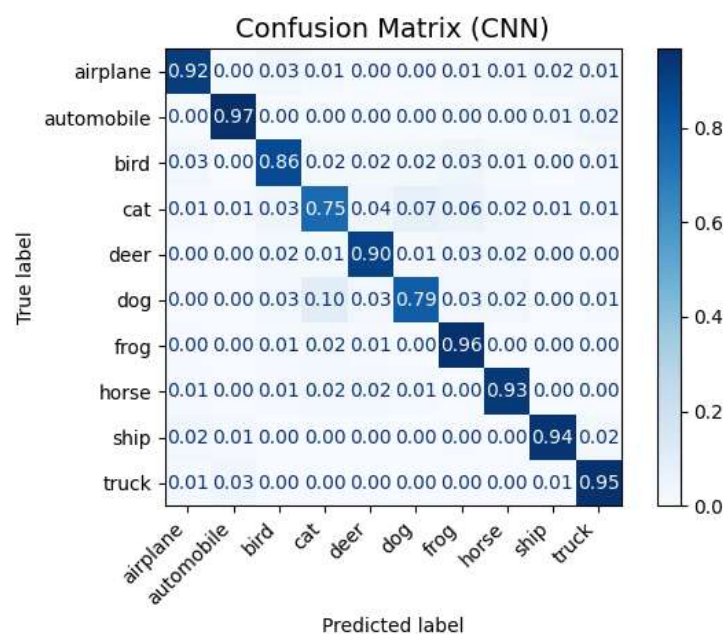
cross-entropy loss με label smoothing. Εφαρμόζουμε και πάλι 80 εποχές εκπαίδευσης. Τα αποτελέσματα που προκύπτουν είναι τα εξής:

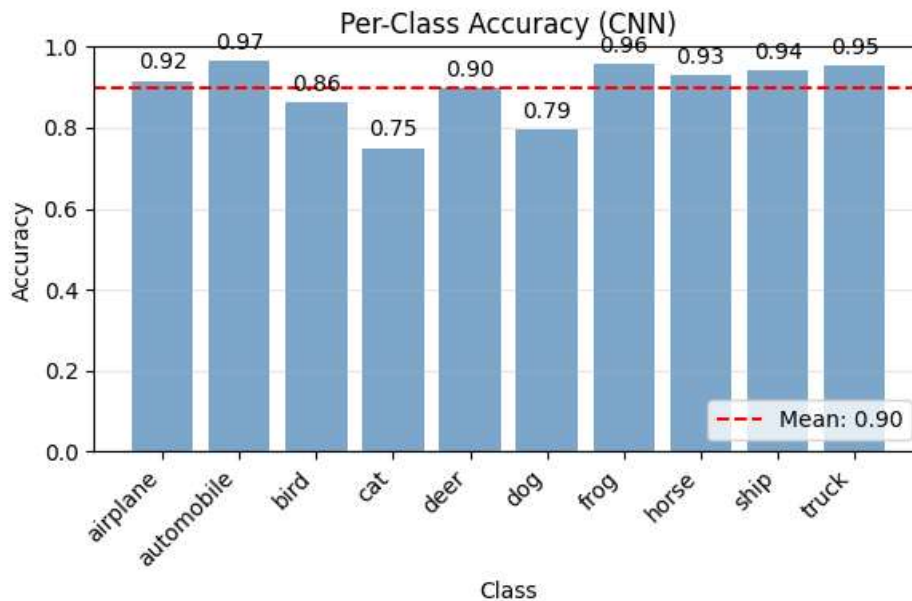
- Training time: 6521.26 seconds
- Test accuracy: 89.72%
- Test loss: 0.7582
- Model stopped at epoch 57, restoring weights from epoch 52



Τα αποτελέσματα αυτά δείχνουν ένα δυνατό μοντέλο με training accuracy έως και 96% και validation accuracy γύρω στο 90%, που είναι ρεαλιστικό ταβάνι για custom CNN στο CIFAR-10. Το κενό αυτό δείχνει ένα λογικό overfitting, αλλά όχι καταστροφικό, απλώς το μοντέλο ταιριάζει πολύ καλά το training set. Η συμπεριφορά αυτή είναι τυπική για deep CNN σε CIFAR-10 με data augmentation. Το γεγονός ότι το training/validation loss δεν μειώνεται όσο πριν είναι επίσης δικαιολογημένο λόγω του label smoothing.

Παρατίθενται αναλυτικά αποτελέσματα ανά κλάση για το τελικό μοντέλο:





3.3 Σύγκριση – Συμπεράσματα

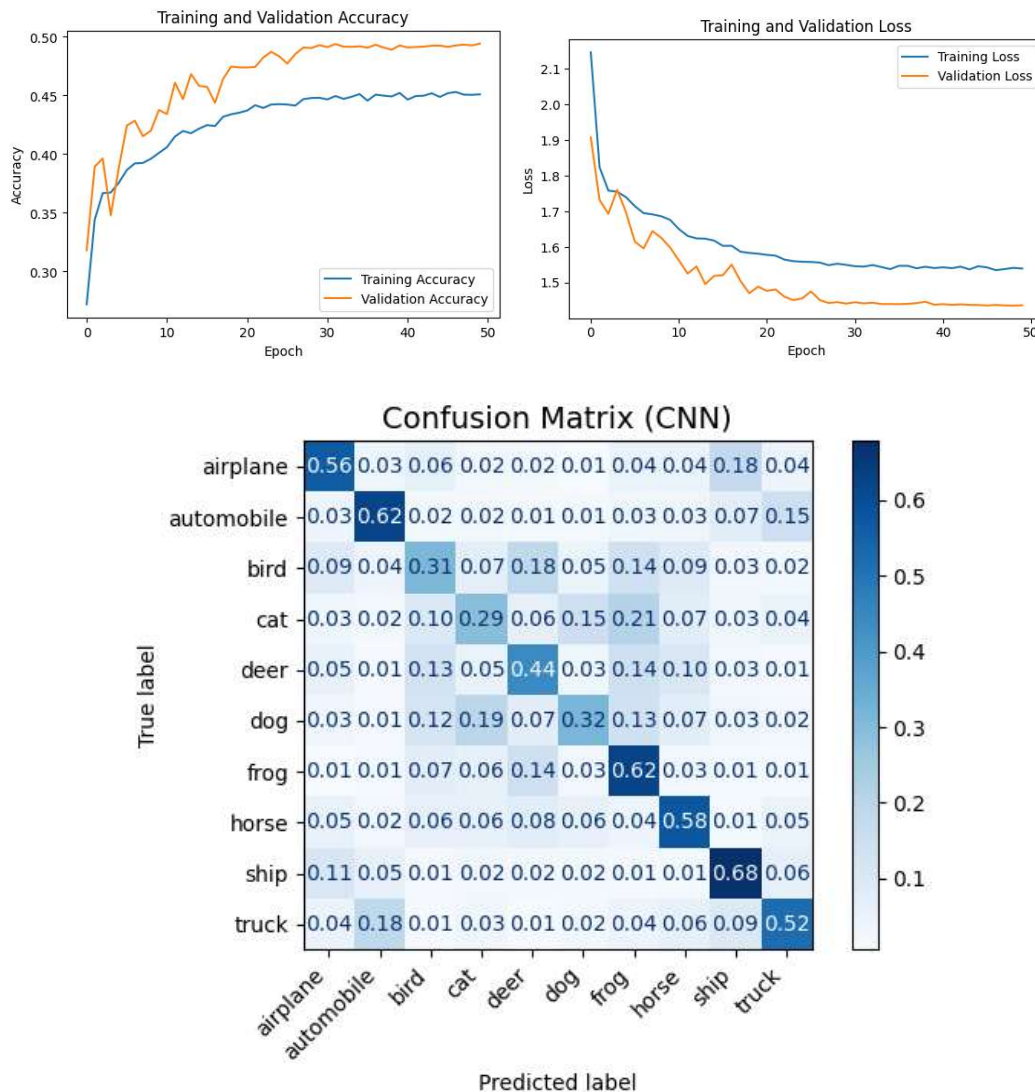
Συνοψίζοντας, η εκπαίδευση του συνελικτικού νευρωνικού δικτύου οδήγησε από ένα απλό base model με test accuracy γύρω στο 70% σε ένα βαθύ μοντέλο με Global Average Pooling, Batch Normalization, Data Augmentation, Early Stopping, δυναμική μείωση του Learning Rate και custom Cross-entropy loss με label smoothing, το οποίο πέτυχε τελικό test accuracy 89.72% με training accuracy περίπου 96% και ήπιο, ελεγχόμενο overfitting στις καμπύλες εκμάθησης. Σε αντίθεση με τις κλασικές μεθόδους kNN και NCC, όπου ακόμη και η βέλτιστη ρύθμιση (kNN με μετρική Cosine και $k = 3$) δεν ξεπέρασε περίπου το 43.5% test accuracy και ο NCC κινήθηκε μόλις στο 28% – 29%, το CNN υπερδιπλασιάζει την ακρίβεια ταξινόμησης στο CIFAR-10, αποδεικνύοντας ότι η αυτόματη εξαγωγή χωρικών χαρακτηριστικών μέσω συνελίξεων είναι η καταλληλότερη προσέγγιση για πολύπλοκα δεδομένα εικόνας. Παρόλο που το κόστος εκπαίδευσης του 10ου μοντέλου είναι σημαντικά μεγαλύτερο, το τελικό δίκτυο πραγματοποιεί γρήγορη πρόβλεψη και παρουσιάζει αξιοσημείωτα καλή ακρίβεια.

4 Μέρος Γ – Multilayer Perceptron

Από άποψη καθαρού ενδιαφέροντος, ορισμένες από τις τεχνικές που μας οδήγησαν στο βέλτιστο μοντέλο CNN, εφαρμόστηκαν και σε ένα MLP μοντέλο. Τα αποτελέσματα που προκύπτουν είναι τα εξής:

- MLP training time: 640.59 seconds
- MLP test accuracy: 49.49%

- MLP test loss: 1.4213
- Model did not early stopped, restoring weights from epoch 49



Το MLP επιτυγχάνει σαφώς χαμηλότερη απόδοση από το CNN: το τελικό test accuracy είναι περίπου 49.5%, ενώ το βέλτιστο CNN φτάνει περίπου 89.7%, άρα το συνελκτικό δίκτυο σχεδόν διπλασιάζει την ακρίβεια στο CIFAR-10. Στις καμπύλες του MLP, τόσο το training όσο και το validation loss παραμένουν σχετικά υψηλά, ενώ οι ακρίβειες σταθεροποιούνται γύρω στο 45% train και 49% validation, με το validation μάλιστα να είναι σταθερά καλύτερο από το training. Αυτό δείχνει ότι το MLP αδυνατεί να εκμεταλλευτεί πλήρως τη δομή των εικόνων, κάτι που φαίνεται και από το confusion matrix όπου πολλές κλάσεις (ιδίως ζώα) μπερδεύονται έντονα μεταξύ τους. Συνεπώς, για τις συγκεκριμένες τεχνικές εκπαίδευσης και regularization, το αποτέλεσμα δείχνει ξεκάθαρα ότι για δεδομένα εικόνες όπως το CIFAR-10 η συνελκτική αρχιτεκτονική είναι δραματικά πιο κατάλληλη από ένα MLP πάνω σε απλά «flattened» pixels. Για καλύτερη απόδοση μέσω MLP μοντέλου απαιτείται τελείως διαφορετική αρχιτεκτονική και τεχνικές εκπαίδευσης.